

DDRoute: a Novel Depth-Driven Approach to the Qubit Routing Problem

Alessandro Annechini
Politecnico di Milano

Milan, ITA

alessandro.annechini@mail.polimi.it

Marco Venere
Politecnico di Milano

Milan, ITA

marco.venere@polimi.it

Donatella Sciuto
Politecnico di Milano

Milan, ITA

donatella.sciuto@polimi.it

Marco D. Santambrogio
Politecnico di Milano

Milan, ITA

marco.santambrogio@polimi.it

Abstract—In the Noisy Intermediate-Scale Quantum (NISQ) era, the topological constraints present in many of the currently available quantum devices pose a physical limit on the feasible interactions between qubits. To comply with such limitations, the compilation of quantum circuits requires solving the Qubit Routing Problem (QRP), by inserting SWAP operations among qubits. The State of the Art provides heuristic algorithms addressing this task, yet the depth of the output circuits is often incompatible with the current limits of quantum hardware. Therefore, we propose DDRoute, a novel heuristic algorithm to solve QRP, designed to reduce the depth overhead introduced by the routing process in the compiled circuits. Our experimental evaluation proves the efficiency of our approach, with a depth reduction of up to 70% with respect to the state-of-the-art routing procedures.

Index Terms—Quantum Computing, Qubit Mapping, Qubit Routing, Quantum Circuit, Depth Minimization.

I. INTRODUCTION

Quantum computing has raised particular interest in the last decades due to its potential in different relevant applications. Quantum algorithms are able to solve important problems with remarkable speedups over their classical counterparts, such as integer factorization [1] and unstructured database search [2]. The availability of quantum computing devices has by now become a reality, with IBM [3], Rigetti [4], and Google [5], among others, releasing powerful quantum processors. Current devices offer hundreds of quantum bits (qubits), but their computational capabilities are limited due to the connectivity constraints between qubits in the Quantum Processing Unit (QPU) [6]. Moreover, further technological issues lie between the development of quantum software and its implementation on quantum hardware, such as the limited fidelity of the operations and the restricted coherence time of each qubit, which introduce errors in the computation and limit the maximum allowed execution time [7].

To ease development, current software toolchains allow engineers to represent quantum algorithms as abstract circuits, based on a high-level instruction set (referred to as *gates* of the circuit) and *logical* qubits with unlimited connectivity. This abstraction layer does not consider hardware limitations, and relies on compilation to produce hardware-compliant quantum assembly. Such a process resembles classical Electronic Design Automation, since it requires Place & Route algorithms to map high-level logical qubits to the *physical* qubits belonging

to the machine, and to *route* such qubits, to overcome their connectivity issues and implement multi-qubit operations.

More specifically, the Qubit Routing Problem (QRP) considers the connectivity constraints among qubits. To execute a two-qubit operation between two unconnected physical qubits, it is necessary to modify their mapping, by inserting SWAP (or equivalent) gates along the circuit. Such a process may increment both the number of two-qubit operations and the depth of the final circuit, thus limiting reliability and support from current NISQ architectures. In particular, the insertion of SWAP operations decreases the fidelity of the circuit, due to the high error rates of physical implementations of gates. Furthermore, the increase in depth due to the addition of SWAPs may also produce circuits that are incompatible with the coherence time of the physical qubits. Algorithms addressing QRP should, therefore, minimize both the number of SWAP operations and the overall circuit depth. Unfortunately, these two objectives represent NP-complete tasks [8], [9], and the State of the Art provides heuristics [10]–[14], which often trade-off optimality for compilation runtime. As a consequence, output circuits are often excessively deep, thus exceeding the allowed limits to maintain coherence.

In this work, we propose DDRoute¹, a depth-driven *routing* algorithm with a heuristic approach that aims to reduce the overhead in circuit depth introduced by the routing process. We employ a custom min-path procedure, which exploits a *generalized distance function* between qubits to schedule SWAP gates efficiently, allowing several quantum operations to be executed in parallel, in order to reduce the final depth of the output quantum circuits. Finally, we introduce DDPlace, a qubit *mapping* technique which provides convenient initial conditions for DDRoute.

Our work aims to provide *flexibility* with respect to the specific topology of the QPU; *scalability*, to maintain reduced compile time even in presence of hundreds of logical qubits; and *gate length awareness*, meaning that the instruction scheduling is sensitive to the duration of the supported gates on the QPU. We theoretically evaluate the computational complexity of DDRoute, and we provide experimental validation, demonstrating that we achieve up to 70% reduction in depth with respect to state-of-the-art routing algorithms.

¹Our work is available at <https://github.com/necst/ddroute>

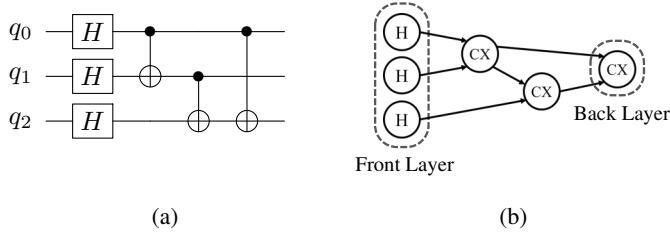


Fig. 1: (a) Quantum circuit describing 3 logical qubits (i.e., q_0 , q_1 , and q_2), 3 Hadamard gates and 3 CNOT/CX gates. (b) Corresponding DAG representation, highlighting front and back layer of the circuit.

II. PRELIMINARIES

In this Section, we provide an introduction to quantum computing and QRP. We specifically refer to the gate-based model of quantum information.

A. Gate-based Model of Quantum Computing

Quantum computing is a paradigm of computation based on the laws of quantum mechanics. Its unit of information is the *qubit*. While classical bits can lie in one of two states, 0 and 1, qubits are described as a *superposition* of two basis states, $|0\rangle$ and $|1\rangle$. The state of a qubit can be represented as a linear combination of the two basis states: $|\Psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where α and β are complex coefficients, and $|\alpha|^2 + |\beta|^2 = 1$ [15]. We can modify a quantum state by applying reversible operations, called *gates*. Current physical implementations of gates support either one operand (single-qubit gate) or two operands (two-qubit gate). Among the most employed single-qubit gates are the Pauli X gate (equivalent to a classical NOT operation) and the Hadamard H gate, often used to create uniform superpositions of basis states; among the most common two-qubit operations are the Controlled-X (CX or CNOT), which takes as input a control qubit and a target qubit and bit-flips the latter if the former is set to $|1\rangle$, and the SWAP operation, employed for swapping the state of two qubits. One SWAP gate can be implemented using three CNOTs, and more generally any n -qubit quantum gate can be decomposed in a set of single-qubit gates and CNOTs [16].

A common representation of quantum algorithms is the *quantum circuit*, where qubits are represented as wires and gates as blocks acting along the wires of their operands. An example of quantum circuit is given in Figure 1a. For circuit compilation, a more convenient representation is the Directed Acyclic Graph (DAG), where nodes represent gates, and edges represent dependencies between gates. We define the front layer F of a DAG as the set of nodes without incoming arcs, corresponding to the operations that can be executed without waiting for previous dependencies. The back layer B , instead, is the set of nodes without outgoing arcs. An example of DAG for the circuit in Figure 1a is shown in Figure 1b.

We now introduce three distinct concepts of *depth*, acting at different levels of the quantum algorithm. The *gate depth*

of a node in a DAG is the length of the longest path between any of the nodes in the front layer F and the given node. As an example, the first CX from the left in Figure 1b has a gate depth equal to 1, since both its operands belong to F . The *qubit depth* of a given qubit, instead, is $1 +$ the length of the longest path between any of the nodes in F and the node corresponding to the last operation applied to that qubit. This can be interpreted as the number of *time steps* required to execute all the operations on a specific qubit, assuming that independent operations are performed in parallel, when possible. In Figure 1a, q_0 and q_2 have qubit depth equal to 4, while q_1 has qubit depth equal to 3. Finally, the *circuit depth* of a given circuit is defined as the maximum among the qubit depths of all the qubits belonging to that circuit. This can be interpreted as the number of time steps required to completely execute the given circuit. For the circuit in Figure 1b, the overall circuit depth is equal to $\max\{4, 3, 4\} = 4$.

B. The Qubit Routing Problem

Current NISQ devices mainly support only single-qubit and two-qubit operations, with limited gate set and connectivity. Therefore, in order to compile a generic n -qubit gate to a specific device backend, this operation is decomposed in single-qubit and two-qubit gates which are directly supported by the platform. After gate decomposition, the n logical qubits are associated with physical qubits on the device (*mapping* or *placement*), and operations are scheduled accordingly [17]. This mapping can be mathematically described by a function $\pi : q \mapsto \tilde{q}$, mapping a logical qubit q to a physical qubit $\tilde{q}$.

Whilst single-qubit gates can be directly applied onto their operands $\tilde{q}$, two-qubit gates require that their operands are directly connected in the quantum processor. For example, if the two-qubit gate G is applied to logical qubits q_1 and q_2 , then the corresponding physical qubits $\tilde{q}_1$ and $\tilde{q}_2$ must be directly connected. Such topological constraints can be visualized as a 2D *coupling map*, where nodes are physical qubits and edges represent supported interconnections for two-qubit operations [6]. In case such constraints are not respected by the initial mapping π , qubit routing must be performed to modify the physical mapping of q_1 and q_2 , such that their newer mapping $\tilde{q}_1$ and $\tilde{q}_2$ are interconnected.

Assuming that we need to change the physical location of a logical qubit q from $\tilde{q}_a$ to $\tilde{q}_b$, two possible cases arise: if $\tilde{q}_a$ and $\tilde{q}_b$ are directly connected in the coupling map, a SWAP operation between $\tilde{q}_a$ and $\tilde{q}_b$ will be sufficient; alternatively, if $\tilde{q}_a$ and $\tilde{q}_b$ are not directly connected, the routing algorithm must find a path between $\tilde{q}_a$ and $\tilde{q}_b$ in the coupling graph, and schedule SWAP operations accordingly. Identifying a convenient selection of qubits to swap in order to minimize the overall circuit depth corresponds to solving the QRP.

C. Related Works

The literature provides a theoretical analysis of the QRP, showing that both the minimization of the number of SWAP gates [8], useful to improve fidelity, and the minimization of

the circuit depth [9] are NP-complete problems. Therefore, based on the complexity of the instances and the requirements of the users, current approaches to address QRP may prefer reduced execution time or higher accuracy. Thus, two families of algorithms were developed, having different objectives. The first family tackles the problem by formulating all the topological constraints and the quantum operations by means of mathematical constraints or clauses, to be resolved by a SAT solver or other exact procedures [17]–[22]. These algorithms provide accurate solutions, but their exponential time complexity makes their use unfeasible for large circuits. For instance, the exact solution introduced in [17] requires $\mathcal{O}((n!)^2 \cdot n \cdot g)$ steps to solve the QRP on a circuit with n qubits and g two-qubit gates, and other similar approaches only tackle quantum circuits with few qubits and hundreds of operations. The second family, currently in use in modern quantum computing frameworks, follows a heuristic approach, effectively reducing execution time and providing good solutions, but without guaranteeing optimality [10]–[14]. As an example, the approach employed in *t|ket* [12] and Cirq [13] is to discard single-qubit gates and divide the circuit into independent sets of two-qubit gates called *timesteps* or *layers*. Then, the QRP is solved on each timestep, employing a heuristic cost function to select the SWAPs to be inserted. A similar approach is followed by QMAP [14], which employs an A*-search-based procedure to find the best SWAPs, and StochasticSwap [11], which uses a randomized approach to select an efficient permutation to be applied to logical qubits. Following a different direction, the algorithm SABRE [10] does not act on independent timesteps, but performs routing incrementally using a similar heuristic cost function, with a worst-case time complexity of $\mathcal{O}(gn^{2.5})$.

The primary objective of these algorithms is to reduce the number of additional gates, yet *not* to minimize the final depth of the given circuit. Their final solutions often present excessive depth, which is inconvenient for the satisfaction of the coherence time constraints. Moreover, their heuristics often exclude solutions where the addition of further SWAPs may result in a circuit with reduced depth, due to parallelization of operations. We deem that this feature is pivotal for NISQ devices, whose supported circuit depth is still limited [23]. Therefore, DDRoute follows a *depth-driven* approach.

III. OUR PROPOSED ALGORITHM

This Section introduces DDRoute, DDPlace, and an analysis of the computational complexity of both components.

A. DDRoute: Algorithm Overview

Given a logical circuit C_l , the goal of DDRoute is to compute an equivalent physical circuit C_p where all the two-qubit gates comply with the coupling map of the device. To do so, logical operations from C_l are sequentially evaluated and added to C_p , along with the necessary SWAP gates.

The setting conditions of DDRoute, i.e., the initial mapping $\pi(\cdot)$, are provided by DDPlace (Section III-B), and are tailored to reduce the number of SWAPs needed to route the first instructions of C_l . After this initialization, DDRoute begins

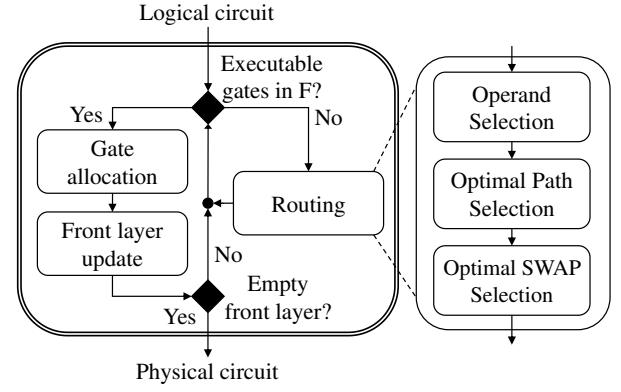


Fig. 2: Overview of DDRoute. The procedure explores F and performs routing when necessary. At every iteration, two-qubit gates are allocated in the physical circuit and F is updated.

by inspecting F , as shown in Figure 2. If one of the gates in F is directly executable, i.e., its operands are mapped to adjacent physical qubits, it can be added to C_p . Afterwards, such a gate is removed from the DAG, and F is updated accordingly. If none of the gates in F can be executed due to topological constraints, it means that the circuit contains two-qubit gates whose operands are not interconnected on the coupling graph. Therefore, SWAP operations have to be scheduled to allow those two-qubit gates to be executed. The criterion for choosing which SWAP operations to perform is pivotal, since it defines the depth of the final circuit. Our SWAP allocation process is based on three main heuristic components: OPERANDSELECTION, OPTIMALPATHSELECTION, and OPTIMALSWAPSELECTION. DDRoute iteratively exploits such components until all the gates in C_l have been correctly routed.

1) *Operand Selection*: The OPERANDSELECTION procedure is responsible for choosing the *operands* of the SWAP gates, i.e., two qubits that need to be routed. The first one is the input of a gate in F whose mapping needs to be modified. The choice of a specific qubit aims to minimize the schedule time of the inserted SWAPs, to reduce the overall circuit depth. To do so, we select the logical operand whose corresponding physical qubit (according to the current mapping) has a low *qubit depth*. Since the depth of a possible SWAP applied to the selected qubit is determined by the qubit depth of both its operands, we improve this metric by also considering the qubit depth of the neighbors of the physical qubit. The one which minimizes these factors is selected as the current operand to be routed. We refer to the selected logical qubit as q_{curr} , to the corresponding gate in F as g_{curr} , and to the other operand as q_{oth} .

2) *Optimal Path Selection*: The OPTIMALPATHSELECTION routine is employed to compute a path in the topology graph that connects the physical qubits corresponding to the operands selected in the previous step. Inserting SWAPs along this path would route the selected logical qubit in a position adjacent to its corresponding operand, making the

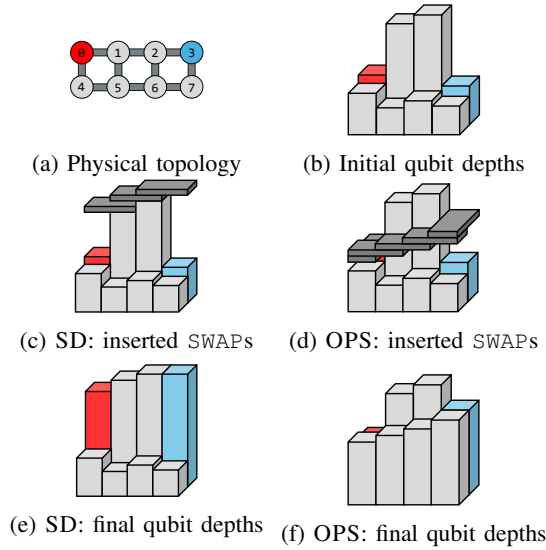


Fig. 3: Shortest Distance (SD) and OPTIMALPATHSELECTION (OPS) on a routing problem between qubit 0 and 3.

operation feasible on hardware. Most algorithms employ a simple shortest-path approach, while the goal of OPTIMALPATHSELECTION is to exploit the same efficient shortest-path procedures, yet with a tailored, depth-driven distance metric. To do so, we define a *generalized distance function* $\delta(\tilde{p})$, able to reduce the depth of each operation along the path and, most importantly, the depth of the final circuit.

More specifically, given a physical source qubit $\tilde{q}$, another physical qubit $\tilde{p}$ in the coupling map, and a path ρ from $\tilde{q}$ to $\tilde{p}$, we define $\delta_{depth}(\tilde{p})$ as the depth of the last SWAP gate along the path ρ ; $\delta_{swap}(\tilde{p})$ as the number of SWAP required to reach $\tilde{p}$; and $\delta_{next}(\tilde{p})$, that is a score depending on the quality of the mapping resulting after applying the SWAP gates required by ρ . In particular, for each of the gates immediately after g_{curr} in the logical circuit, a positive penalty is assigned to SWAP gates that bring its operands further apart and a negative discount is assigned to SWAP gates that bring its operands closer.

Thus, we combine these scores and define the generalized distance $\delta(\tilde{p})$ from $\tilde{q}$ to $\tilde{p}$ as:

$$\delta(\tilde{p}) = \delta_{depth}(\tilde{p}) + \delta_{swap}(\tilde{p}) + \delta_{next}(\tilde{p})$$

This definition allows the distance function to be efficiently computed using Dijkstra's algorithm, with the same computational complexity as a simple shortest-path computation. The algorithm visits one physical qubit at a time, selecting at each step the nearest one not yet visited and updating the shortest path to its neighbors. The procedure terminates when the visited node is the routing destination.

Figure 3 provides an example with a physical topology of eight qubits (3a). We assume that a certain number of gates has already been allocated in C_p , with the qubit depths represented by the height of the corresponding columns (3b). Notice that the *circuit depth* equals to the highest column. The goal is to modify the mapping of an operand from qubit 0 to qubit

3 by means of SWAPs (represented as darker blocks). The simple shortest distance (SD) approach solves the problem by inserting three SWAPs (3c), with an unnecessary increase in the circuit depth (3e). The OPTIMALPATHSELECTION (OPS), instead, follows a seemingly longer path (3d), yet reaching the destination qubit at a lower depth without increasing the overall circuit depth thanks to $\delta(\tilde{p})$ (3f).

3) *Optimal SWAP Selection*: During the OPTIMALPATHSELECTION, the routing algorithm obtains a path ρ between the physical operands $\tilde{q} = \pi(q_{curr})$ and $\tilde{p} = \pi(q_{oth})$. Placing *all* the SWAPs along the path ρ is not, however, the best strategy, since it does not take into account the possible SWAPs that can be performed by other operands in parallel. A simple yet effective strategy to exploit parallelism, employed by the OPTIMALSWAPSELECTION, is to allocate just the first operation along the path, updating the mapping of q_{curr} accordingly. If $\pi(q_{curr})$ and $\pi(q_{oth})$ are now adjacent, meaning that the end of ρ has been reached, then the corresponding gate g_{curr} can be inserted in C_p and removed from F . If $\pi(q_{curr})$ and $\pi(q_{oth})$ are not adjacent, then, instead of inserting the remaining SWAPs, the routing process is restarted from the OPERANDSELECTION step. Then, if the selected operand is still q_{curr} , the optimal path computation is not required, as the best operation to be inserted is the next SWAP in the previously computed path ρ . If the selected operand is different, instead, the previous optimal path is discarded and OPTIMALPATHSELECTION is performed again. This assures that all the SWAPs added to C_p are associated with the current best operand, thus reducing the schedule time of all the inserted SWAPs. Once all the gates in C_l have been inserted in C_p , causing F to be empty, the routing process can terminate and C_p can be executed on the QPU.

Finally, a key aspect is the allocation of single-qubit gates. Most routing algorithms ignore the presence of single-qubit operations as they are not subject to topological constraints, inserting them in the circuit after the routing procedure. To reduce their depth contribution, DDRoute postpones their allocation until a two-qubit gate, i.e., an operation from C_l or a routing SWAP, has to be inserted. In the first case, it has no choice but to insert all the postponed single-qubit gates right before the new operation. In the case of a SWAP, instead, it can exploit the fact that the SWAP gate depth depends on the highest qubit depth among its operands. If the physical operand with the lowest qubit depth corresponds to the physical mapping of a logical qubit, then the routing procedure inserts single-qubit gates acting on that logical qubit before the new SWAP, until it fills the difference in qubit depth between the two qubits or it meets a two-qubit gate or the end of the circuit. Using this strategy, the algorithm can minimize the contribution of single-qubit gates to the final circuit depth.

B. DDPlace

DDPlace is our depth-driven procedure to compute the initial mapping $\pi(\cdot)$ for DDRoute. It begins by sorting all the two-qubit gates in C_l by depth. Then, it selects two random physical qubits and maps the operands of the first two-qubit

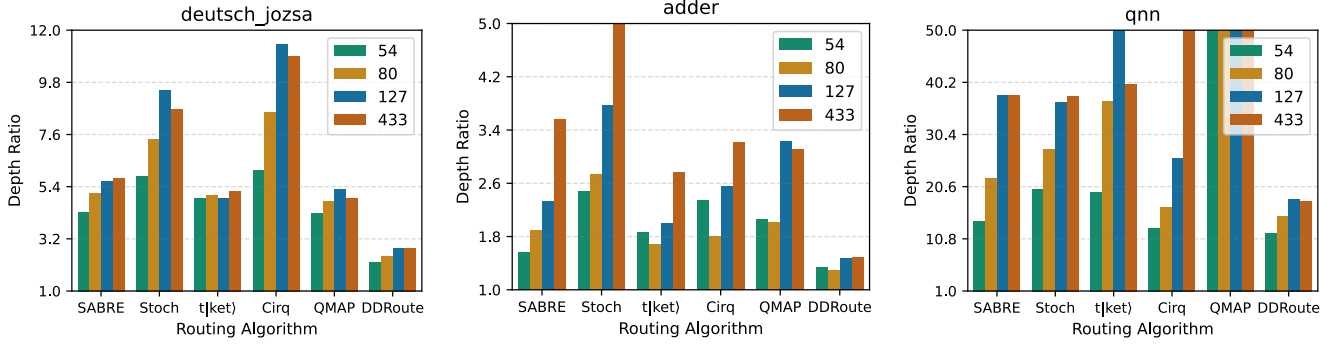


Fig. 4: Depth ratio comparison on DEUTSCH_JOSZA, ADDER, and QNN circuits, targeting the 54-qubit Google Sycamore, 80-qubit Rigetti Aspen, 127-qubit IBM Eagle, and 433-qubit IBM Osprey topologies.

gate onto them. The procedure then iterates over the remaining two-qubit gates, ordered by depth, mapping their remaining operands. If both the operands have not been mapped yet, they are assigned to two physical qubits which are the closest to the last assigned physical qubit. If one of the two operands has not been mapped yet, it is associated with the *free* physical qubit closest to the one assigned to the other operand. Finally, if some logical qubits have not been mapped, they are assigned to the physical qubits closest to the last assigned physical qubit.

C. Computational Complexity

The computational complexity of DDPlace is $\mathcal{O}(g \log n)$, where g and n are the number of two-qubit gates and the number of logical qubits in the circuit, respectively. Indeed, computing the depth of all the two-qubit gates in the graph requires $\mathcal{O}(g)$ operations. Then, F is continuously updated using a binary min-heap. Since the number of two-qubit gates in F at each step is at most $\lfloor \frac{n}{2} \rfloor$, removing the gate with the lowest depth and updating F requires $\mathcal{O}(\log n)$ operations. In the worst case, when the whole circuit traversal is necessary, DDPlace requires $\mathcal{O}(g \log n)$ operations.

The computational complexity of DDRoute is $\mathcal{O}(gn^{1.5} \log n)$. Indeed, on a 2D coupling map, the average number of SWAPs required to map two operands of a gate onto adjacent physical qubits is $\mathcal{O}(\sqrt{n})$, resulting in a total of $\mathcal{O}(g\sqrt{n})$ SWAP insertions. In the worst case, the selected operand q_{curr} is changed for each SWAP, causing the recomputation of the optimal path ρ , where $\mathcal{O}(n \log n)$ steps are required by the OPTIMALPATHSELECTION procedure. Therefore, the complexity of the whole algorithm is $\mathcal{O}(g\sqrt{n}(n \log n)) = \mathcal{O}(gn^{1.5} \log n)$.

IV. EXPERIMENTAL RESULTS

In this Section, we experimentally evaluate DDRoute, and its DDPlace initialization, comparing its performance against other state-of-the-art algorithms. We consider multiple benchmark suites and target superconducting architectures.

A. Experimental Setup

Our evaluations aim to compare the performance of DDRoute, initialized by DDPlace, against other state-of-the-

art procedures. The algorithms we use for comparison are SABRE [10] with Qiskit v1.2.0, t|ket [12] v1.31.0, Google Cirq [13] v1.4.1, and MQT QMAP [14] v2.6.0, initialized by their default mapping technique; and StochasticSwap [11], with Qiskit DenseLayout [24] as a mapping procedure. We consider state-of-the-art benchmark suites: the Munich Quantum Toolkit Benchmark [25], QASMBench [26] v1.4, VeriQBench [27], and Qiskit Circuit Library [28], and target the following QPUs: Google Sycamore [29] (54 qubits), Rigetti Aspen [30] (80 qubits), IBM Eagle [31] (127 qubits), IBM Osprey [32] (433 qubits), and IBM Condor [33] (1121 qubits). All our experiments were executed on a 32-core Intel Xeon E5-2680 CPU with 32GB of memory and the Ubuntu 24.04 LTS operating system, setting a compile time limit of 3 hours. We perform three classes of testing: a depth analysis, a scaling analysis, and an analysis on gate length awareness.

B. Depth Analysis on Benchmark Circuits

This class of experiments compares DDRoute against the State of the Art, targeting medium-sized circuits and QPUs, with a focus on the depth overhead introduced by the routing. To provide more accurate results, we run all stochastic algorithms 30 times, and we average depth and compile time values. We define the *depth ratio* of circuit c as:

$$\text{DepthRatio}[c] = \frac{\text{FinalDepth}[c]}{\text{InitialDepth}[c]}$$

which quantifies the overhead of the routing process on the final depth. Indeed, the addition of SWAP gates produces outputs with increased depth with respect to the input circuit. The more the routing procedure is efficient, the closer the depth ratio to 1. Additionally, we define the improvement in depth of DDRoute with respect to a generic algorithm ALG for circuit c as:

$$\text{Improvement}[c] = \frac{\text{FinalDepth}_{\text{ALG}}[c] - \text{FinalDepth}_{\text{DDRoute}}[c]}{\text{FinalDepth}_{\text{ALG}}[c]}$$

The results of our experiments, such as the ones presented in Figure 4, show a consistent reduction in circuit depth. DDRoute is able to outperform all the algorithms employed in the current quantum computing frameworks, both in depth

TABLE I: Improvement of DDRoute with respect to the State of the Art, targeting the 80-qubit Rigetti Aspen, the 127-qubit IBM Eagle, and the 433-qubit IBM Osprey topology.

#Qubits	Improvement (%) of DDRoute w.r.t.				
	SABRE	Stochastic	t ket	Cirq	QMAP
54	25.52	46.98	39.13	26.14	51.62
80	34.93	47.17	41.51	29.05	55.10
127	36.06	53.87	47.41	45.27	70.86
433	47.97	58.28	47.53	60.68	71.15
Average	36.88	52.02	43.02	40.92	62.56

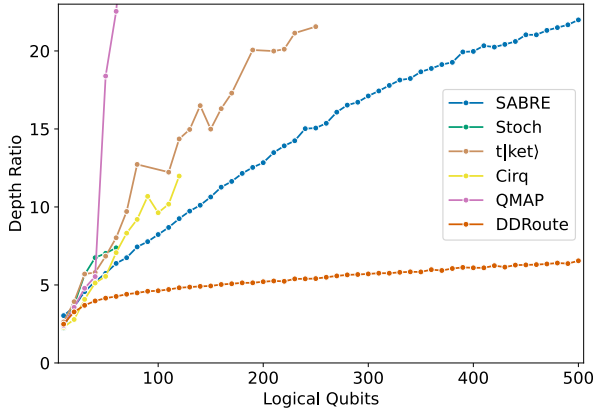


Fig. 5: Depth ratio comparison on circuits generated with Qiskit Circuit Library, targeting the 1121-qubit IBM Condor.

and compile time, with an average depth improvement of over 36.8% and a geometric average speedup of $92.8\times$ over all experiments. We point out that the achieved speedup in compilation time also depends on the execution runtime of the algorithms, which is based on Python and C++ for DDRoute, and mainly on Python for the State of the Art. Regarding the SWAP count of the output circuits, we note that DDRoute is outperformed by algorithms like SABRE and QMAP, where the minimization of the number of added operations is the main target objective, rather than circuit depth. Table I shows a detailed comparison of DDRoute with the six algorithms on circuits and topologies with different sizes. We highlight that the increase in circuit size and physical qubits produces up to 71.15% improvement.

C. Scalability Analysis & Awareness of Gate Length

To further quantify the scalability trend, we performed another set of experiments targeting the 1121-qubit IBM Condor topology. We generated input circuits combining several components of the Qiskit Circuit Library [28], from 10 to 500 qubits. The results of these experiments are shown in Figure 5. Out of the six compared algorithms, only DDRoute and SABRE were able to complete routing for all circuits without exceeding the time limit of 3 hours. As it can be seen, DDRoute outperforms SABRE, with an average depth ratio of 6.51 against 21.99 for the largest test case, reducing the circuit depth by 70.4%. Regarding the count of SWAPs, which is *not*

TABLE II: Depth comparison with different metrics between SABRE and DDRoute, on circuits generated with Qiskit Circuit Library, targeting the 1121-qubit IBM Condor.

#Qubits	Metric	SABRE	DDRoute	Improvement (%)
100	1-1-3	43065	24206	43.79
	1-2-6	82082	46171	43.75
	1/0-11-33	430457	241768	43.83
300	1-1-3	270919	90509	66.59
	1-2-6	530601	172695	67.45
	1/0-11-33	2853356	919159	67.79
500	1-1-3	581238	172020	70.40
	1-2-6	1143512	329845	71.16
	1/0-11-33	6183399	1761279	71.52

the main target of DDRoute, SABRE only achieves 38.7% improvement over DDRoute in the largest test case. Therefore, DDRoute proves to provide an important reduction in circuit depth with only a limited increase in SWAPs.

As a final class of experiments, we aim to evaluate the awareness of gate length, i.e., the ability of the routing procedure to take into account $\tau(g)$, corresponding to the duration of each gate g , and minimize the circuit depth accordingly. To perform such evaluation, we repeated the experiments on the largest available topology, IBM Condor, varying the metric over which the circuit depth is computed. We employed three different metrics for $\tau(g)$: **1-1-3**, where the depth of single-qubit gates, CXs, and SWAPs are 1, 1, and 3, respectively; **1-2-6**, where the depth of single-qubit gates, CXs, and SWAPs are 1, 2, and 6 [34]; and **1/0-11-33**, where X and SX gates have a depth of 1, RZs have a depth of 0, CXs of 11, and SWAP operations of 33. The latter is proportional to the average gate execution time on IBM devices [35], [36]. Table II shows a comparison for some of the test cases between SABRE and DDRoute, highlighting that DDRoute achieves significant improvements in the final depth for all three metrics, with the ones reflecting different gate durations having an even more important reduction, up to 71.52%. For further flexibility, we also provide the possibility of specifying different $\tau(g, \tilde{q}_1, \tilde{q}_2)$ for different operands of the same gate.

V. CONCLUSION

In this paper, we proposed DDRoute, a depth-driven approach to the QRP, which compiles logical quantum circuits into physical ones, while reducing the overall depth overhead. We devised a generalized distance function, able to quantify the impact of SWAP insertions on the final circuit depth. We also introduced DDPlace, a depth-driven mapping procedure to generate convenient initial conditions for the routing algorithm. Experimental results show that our approach is able to effectively reduce the depth of the routed circuits, achieving up to 70% improvement over state-of-the-art solutions.

As the next steps, we aim to tune the definition of the generalized distance function to keep into consideration additional factors, such as the hardware-specific tradeoff between SWAP fidelity and coherence time.

REFERENCES

- [1] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, ser. SFCS '94. USA: IEEE Computer Society, 1994, p. 124–134. [Online]. Available: <https://doi.org/10.1109/SFCS.1994.365700>
- [2] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, ser. STOC '96. New York, NY, USA: Association for Computing Machinery, 1996, p. 212–219. [Online]. Available: <https://doi.org/10.1145/237814.237866>
- [3] IBM Quantum. (2016) IBM Quantum. IBM. Accessed: 2024-08-21. [Online]. Available: <https://www.ibm.com/quantum>
- [4] Rigetti Computing. (2013) Rigetti Computing. Rigetti Computing. Accessed: 2024-08-21. [Online]. Available: <https://www.rigetti.com/>
- [5] Google Quantum AI. (2013) Google Quantum AI. Google Quantum AI. Accessed: 2024-08-21. [Online]. Available: <https://quantumai.google>
- [6] J. Kusyk, S. M. Saeed, and M. U. Uyar, "Survey on quantum circuit compilation for noisy intermediate-scale quantum computers: Artificial intelligence to heuristics," *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 1–16, 2021.
- [7] S. Resch and U. R. Karpuzcu, "Benchmarking quantum computers and the impact of quantum noise," *ACM Comput. Surv.*, vol. 54, no. 7, Jul. 2021. [Online]. Available: <https://doi.org/10.1145/3464420>
- [8] T. Ito, N. Kakimura, N. Kamiyama, Y. Kobayashi, and Y. Okamoto, "Algorithmic theory of qubit routing," in *Algorithms and Data Structures*, P. Morin and S. Suri, Eds. Cham: Springer Nature Switzerland, 2023, pp. 533–546.
- [9] D. B. Tan and J. Cong, "Optimality study of existing quantum computing layout synthesis tools," *IEEE Transactions on Computers*, vol. 70, no. 09, pp. 1363–1373, 9 2021.
- [10] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for nisq-era quantum devices," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 1001–1014. [Online]. Available: <https://doi.org/10.1145/3297858.3304023>
- [11] Qiskit. (2017) StochasticSwap. IBM Quantum. Accessed: 2024-08-21. [Online]. Available: <https://docs.quantum.ibm.com/api/qiskit/qiskit.transpiler.passes.StochasticSwap>
- [12] S. Sivarajah, S. Dilkes, A. Cowtan, W. Simmons, A. Edgington, and R. Duncan, "t[ket]: A retargetable compiler for NISQ devices," *Quantum Science and Technology*, 2020. [Online]. Available: <https://doi.org/10.1088/2058-9565/ab8e92>
- [13] Quantum AI team and collaborators, "Cirq," 10 2020. [Online]. Available: <https://doi.org/10.5281/zenodo.4062499>
- [14] R. Wille and L. Burgholzer, "Mqt qmap: Efficient quantum circuit mapping," in *Proceedings of the 2023 International Symposium on Physical Design*, ser. ISPD '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 198–204. [Online]. Available: <https://doi.org/10.1145/3569052.3578928>
- [15] M. A. Nielsen and I. L. Chuang, *Quantum computation and quantum information*. Cambridge university press, 2010.
- [16] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. A. Smolin, and H. Weinfurter, "Elementary gates for quantum computation," *Phys. Rev. A*, vol. 52, pp. 3457–3467, 11 1995.
- [17] M. Y. Siraichi, V. F. d. Santos, C. Collange, and F. M. Q. Pereira, "Qubit allocation," in *Proceedings of the 2018 international symposium on code generation and optimization*, 2018, pp. 113–125.
- [18] G. Nannicini, L. S. Bishop, O. Günlük, and P. Jurcevic, "Optimal Qubit Assignment and Routing via Integer Programming," *ACM Transactions on Quantum Computing*, vol. 4, no. 1, 10 2022. [Online]. Available: <https://doi.org/10.1145/3544563>
- [19] A. Molavi, A. Xu, M. Diges, L. Pick, S. Tannu, and A. Albarghouthi, "Qubit mapping and routing via MaxSAT," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 1078–1091.
- [20] P. Zhu, X. Cheng, and Z. Guan, "An exact qubit allocation approach for NISQ architectures," *Quantum Information Processing*, vol. 19, no. 11, p. 391, 11 2020. [Online]. Available: <https://doi.org/10.1007/s11228-020-02901-4>
- [21] A. Shafaei, M. Saeedi, and M. Pedram, "Qubit placement to minimize communication overhead in 2D quantum architectures," in *2014 19th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2014, pp. 495–500.
- [22] C. Zhang, A. B. Hayes, L. Qiu, Y. Jin, Y. Chen, and E. Z. Zhang, "Time-optimal qubit mapping," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 360–374. [Online]. Available: <https://doi.org/10.1145/3445814.3446706>
- [23] IBM Quantum. (2024) Development Roadmap. IBM Quantum. Accessed: 2024-09-12. [Online]. Available: <https://www.ibm.com/quantum/technology#roadmap>
- [24] Qiskit. (2017) DenseLayout. IBM Quantum. Accessed: 2024-08-21. [Online]. Available: <https://docs.quantum.ibm.com/api/qiskit/qiskit.transpiler.passes.DenseLayout>
- [25] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking Software and Design Automation Tools for Quantum Computing," *Quantum*, vol. 7, p. 1062, Jul. 2023. [Online]. Available: <https://doi.org/10.22331/q-2023-07-20-1062>
- [26] A. Li, S. Stein, S. Krishnamoorthy, and J. Ang, "Qasmbench: A low-level quantum benchmark suite for nisq evaluation and simulation," *ACM Transactions on Quantum Computing*, vol. 4, no. 2, feb 2023. [Online]. Available: <https://doi.org/10.1145/3550488>
- [27] K. Chen, W. Fang, J. Guan, X. Hong, M. Huang, J. Liu, Q. Wang, and M. Ying, "VeriQBenchmark: A Benchmark for Multiple Types of Quantum Circuits," 2022. [Online]. Available: <https://arxiv.org/abs/2206.10880>
- [28] Qiskit. (2017) Qiskit Circuit Library. IBM Quantum. Accessed: 2024-08-21. [Online]. Available: https://docs.quantum.ibm.com/api/qiskit/circuit_library
- [29] F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barends, R. Biswas, S. Boixo, F. G. Brandao, D. A. Buell *et al.*, "Quantum supremacy using a programmable superconducting processor," *Nature*, vol. 574, no. 7779, pp. 505–510, 2019.
- [30] Rigetti Computing. (2022) Rigetti Aspen. Rigetti Computing. Accessed: 2024-08-21. [Online]. Available: <https://investors.rigetti.com/news-releases/news-release-details/rigetti-computing-announces-commercial-availability-80-qubit>
- [31] IBM Quantum. (2021) IBM Eagle. IBM Quantum. Accessed: 2024-08-21. [Online]. Available: <https://www.ibm.com/quantum/blog/127-qubit-quantum-processor-eagle>
- [32] —. (2022) IBM Osprey. IBM Quantum. Accessed: 2024-08-21. [Online]. Available: <https://newsroom.ibm.com/2022-11-09-IBM-Unveils-400-Qubit-Plus-Quantum-Processor-and-Next-Generation-IBM-Quantum-System-Two>
- [33] —. (2023) IBM Condor. IBM Quantum. Accessed: 2024-08-21. [Online]. Available: <https://www.ibm.com/quantum/blog/quantum-roadmap-2033>
- [34] H. Deng, Y. Zhang, and Q. Li, "Codar: a contextual duration-aware qubit mapping for various nisq devices," in *Proceedings of the 57th ACM/EDAC/IEEE Design Automation Conference*, ser. DAC '20. IEEE Press, 2020.
- [35] Qiskit. (2017) BackendProperties. IBM Quantum. Accessed: 2024-08-21. [Online]. Available: <https://docs.quantum.ibm.com/api/qiskit/qiskit.providers.models.BackendProperties>
- [36] —. (2017) RZGate. IBM Quantum. Accessed: 2024-08-21. [Online]. Available: <https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.RZGate>